

Trabalho Prático 01

Disciplina Algoritmos I

Raphaela Maria Costa e Silva - 2020006973

Departamento de Ciência da Computação – Universidade Federal de Minas Gerais

Belo Horizonte – MG – Brasil

rapha470@ufmg.com

1. Apresentação

Essa documentação tem como objetivo explicar o problema de maneira sucinta, explicar as estruturas e os algoritmos usados para solucionar o problema, analisar a complexidade de tempo do código.

A proposta do TP foi implementar um programa que retorna a melhor opção de pareamento entre bicicletas e visitantes considerando a nota dos visitantes para cada bicicleta e a distância dos visitantes em relação a cada bicicleta.

A entrada do problema consiste no número de bicicletas e visitantes (que é igual). O tamanho do mapa, e o mapa, que foi recebido como uma matriz de caracteres. E por fim as notas dos visitantes em relação a todas as bicicletas.

Para resolver o problema usei busca em largura para calcular as distâncias entre as bicicletas e os visitantes e o algoritmo de Gale-Shapley para fazer o pareamento considerando que a preferencia de um visitante por uma bicicleta é mais importante que a distância.

2. Modelagem Computacional

A implementação do programa teve como base as estruturas de vector da biblioteca vector e de matriz. Foram escolhidas essas estrutura pela facilidade de iteração dentro delas e pelas funções que são aplicáveis nelas, como por exemplo a função `std::sort` facilitou a escrita do código.

Para guardar o mapa recebido, foi usado uma matriz de tamanho `[MAX][MAX]`, em que MAX é 1000. Isso foi feito assim para facilitar a passagem da matriz por parâmetro na função BFS.

Para guardar a posição das bicicletas no mapa, foi usado uma matriz de tamanho `[qtd][2]`, em que qtd é a quantidade de bicicletas no problema.

Para guardar as preferências dos visitantes foi usado uma array de vector `<pair<int,int>>` `pref`, em que o primeiro elemento do array representa o visitante a, e o segundo elemento representa o visitante b e assim por diante. Já o primeiro elemento do vector é a nota que o visitante deu à bicicleta e o segundo elemento do vector é o id da bicicleta.

Para guardar as preferencias das bicicletas tendo como critério a distância, foi usado um array de vector `<pair<int, int>` `bicPref`. Em que o primeiro elemento do array representa a bicicleta 0, o segundo elemento representa a bicicleta 2 e assim por diante. Já o primeiro

elemento do vetor é a distancia da bicicleta e o sendo vetor é o visitante que está àquela distancia da bicicleta.

Já no Gale-Shapley foram usados além do array de vetor das preferências dos visitantes e do array de vetor das preferências das bicicletas outros dois vectors. O primeiro é o vetor que é guardado os visitantes sem bicicletas. Já o segundo vetor guarda os pares formados de visitantes e bicicletas, que é o vetor que é impresso ao final do programa.

3. Descrição da Solução

Para solucionar o problema foi seguido os seguintes passos:

Receber a entrada. Ao receber o mapa, deixar guardado em uma matriz a posição de cada bicicleta no mapa. Ao receber a preferência dos visitantes, já ordenar usando `std::sort`.

A função `std::sort` ordena as notas em ordem decrescente, em caso empate, essa função ordena o id dos visitantes em ordem decrescente também, então é necessário reordenar em caso de empate.

Para fazer essa reordenação foi usado a função ordenar, que funciona como um bubble sort, mas as trocas são feitas somente se houver empate na nota.

Segue o código da função ordenar em alto nível:

Para todo os elementos no vetor:

 Para todos os elementos do vetor:

 Se houver empate na nota:

 Se o ID do elemento atual for maior que o Id do elemento seguinte:

 Troca o elemento atual com o seguinte;

 Se não tiver nenhuma troca:

 Break;

Fim.

Então é feito uma BFS para cada bicicleta, a BFS retorna um array de vetores com as preferencias das bicicletas já ordenado.

Segue o código da função BFS em alto nível:

Inicializa uma matriz como uma cópia do mapa;

Inicializa um vector vazio para guardar os índices dos vizinhos a visitar;

Inicializa a distância com 0;

Inicializa um vector vazio para guardar a distância e o ID de um visitante.

Enquanto não foi descoberto a distancia para todos os visitantes:

 Incrementa a distância;

 Enquanto não foram visitados todos os vizinhos:

Se algum vizinho do índice no mapa é acessível:

Coloca ele nos vizinhos a visitar;

Se esse vizinho for um visitante:

Coloca a distância e o ID no vector;

Marca esse vizinho como não acessível;

Atualiza a lista para visitar;

Ordena o vector em ordem crescente em relação à distância e ID (caso empate);

Fim.

Em seguida é executado o Gale-Shapley, em que a prioridade é a preferência dos visitantes.

Segue o código do Gale-Shapley em alto nível:

Inicializa o vector Par vazio;

Inicializa o vector semPar com todos os visitantes;

Enquanto algum visitante ainda está sem par:

B recebe a primeira bicicleta ainda não proposta do primeiro visitante de semPar

Se B não tiver par

Guarda o par B e visitante no vector Par;

Apaga visitante de semPar

Se B prefere o visitante do que seu par atual:

Troca o par do B pelo visitante preferido;

Apaga visitante de semPar

Coloca par atual em semPar

Se não:

B rejeita o visitante;

Fim.

Por fim o vector de Par é ordenado e impresso;

4. Análise de complexidade de tempo assintótica

Ignorando a restrição de serem no máximo 10 visitantes e bicicletas e do mapa ter tamanho máximo 1000 x 1000:

A função BFS tem complexidade $O(m*n)$, visto que a função itera no máximo por todos os índices da matriz que forem acessíveis.

A função ordenar usa o método bubble sort, a função tem crescimento assintótico $O(n^2)$ em seu pior caso.

A função preferir precisa iterar no máximo todas as preferências de uma bicicleta. Portanto, tem complexidade $O(n)$.

O Gale-Shapley itera por no máximo todas as preferências dos visitantes e, portanto, tem complexidade $O(n^2)$.

Na função main as funções BFS e ordenar são chamadas n vezes, portanto a função main em seu pior caso é $O(n^3)$. Portanto, o código é $O(n^3)$.

5. Instruções para compilação e execução

O programa foi desenvolvido na linguagem C++, compilada por G++ da GNU Compiler Collection no SO Linux.

Acesse o diretório do arquivo do tp01. Compile os arquivos usando o comando “**g++ -g tp01.cpp -o tp01**”.

Para executar o programa passe em argv[1] o nome do arquivo de entrada, por exemplo: “**tp01 exemplo.txt**”.

O arquivo de entrada deve estar no mesmo diretório do executável.